

The logo of Gazi University is a circular seal. It features the university's name in Turkish, 'GAZİ ÜNİVERSİTESİ', around the top inner edge and the year '1926' at the bottom. In the center is a stylized signature of 'Gazi'.

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 9: Data Manipulation Language-1

CREATE COMMAND-recap

Introduction to PostgreSQL CREATE TABLE statement

Typically, a relational database consists of multiple related tables. Tables allow you to store structured data like customers, products, and employees.

To create a new table, you use the `CREATE TABLE` statement. Here's the basic syntax of the `CREATE TABLE` statement:

```
CREATE TABLE [IF NOT EXISTS] table_name (  
    column1 datatype(length) column_constraint,  
    column2 datatype(length) column_constraint,  
    ...  
    table_constraints  
);
```



CREATE COMMAND- EXAMPLE COMMAND

The following example uses the `CREATE TABLE` statement to create the `accounts` table:

```
CREATE TABLE accounts (  
  user_id SERIAL PRIMARY KEY,  
  username VARCHAR (50) UNIQUE NOT NULL,  
  password VARCHAR (50) NOT NULL,  
  email VARCHAR (255) UNIQUE NOT NULL,  
  created_at TIMESTAMP NOT NULL,  
  last_login TIMESTAMP  
);
```



DATA MANIPULATION OPERATIONS



SELECT, ORDER BY, DISTINCT OPERATIONS

SELECT COMMAND



SELECT

- One of the most common tasks, when you work with the database, is to retrieve data from tables using the SELECT statement.
- The SELECT statement is one of the most complex statements in PostgreSQL. It has many clauses that you can use to form a flexible query.
- Due to its complexity, we will break it down into many shorter and easy-to-understand tutorials so that you can learn about each clause faster.

SELECT

The SELECT statement has the following clauses:

- Select distinct rows using **DISTINCT** operator.
- Sort rows using **ORDER BY** clause.
- Filter rows using **WHERE** clause.
- Select a subset of rows from a table using **LIMIT** or **FETCH** clause.
- Group rows into groups using **GROUP BY** clause.
- Filter groups using **HAVING** clause.
- Join with other tables using joins such as **INNER JOIN**, **LEFT JOIN**, **FULL OUTER JOIN**, **CROSS JOIN** clauses.
- Perform set operations using **UNION**, **INTERSECT**, and **EXCEPT**.

In this class, we are going to focus on the **SELECT** and **FROM** clauses.

SELECT syntax

- Let's start with the basic form of the **SELECT** statement that retrieves data from a single table.
- The following illustrates the syntax of the **SELECT** statement:

```
SELECT
    select_list
FROM
    table_name;
```

In this syntax:

- First, specify a select list that can be a column or a list of columns in a table from which you want to retrieve data. If you specify a list of columns, you need to place a comma (,) between two columns to separate them. If you want to select data from all the columns of the table, you can use an asterisk (*) shorthand instead of specifying all the column names. The select list may also contain expressions or literal values.
- Second, provide the name of the table from which you want to query data after the FROM keyword.

The FROM clause is optional. If you are not querying data from any table, you can omit the FROM clause in the SELECT statement.

SELECT syntax

- Let's open SQL Studio, Create a TABLE then insert some data.
- Once we have some data on the tables, we will use simple query operation to retrieve it.

```
1  -- Create the table
2  CREATE TABLE employees (
3      ssn VARCHAR(11) PRIMARY KEY,
4      fname VARCHAR(50) NOT NULL,
5      minit CHAR(1),
6      lname VARCHAR(50) NOT NULL,
7      birth_date DATE,
8      address TEXT,
9      job_type VARCHAR(50)
10 );
11
12 -- Insert data into the table
13 INSERT INTO employees (ssn, fname, minit, lname, birth_date, address, job_type)
14 VALUES
15 ('987-65-4321', 'John', 'B', 'Smith', '1985-01-15', '123 Maple St, Ankara', 'Engineer'),
16 ('123-45-6789', 'Alice', 'M', 'Johnson', '1992-05-22', '456 Oak Ave, Istanbul', 'Manager'),
17 ('555-00-1212', 'Robert', NULL, 'Taylor', '1978-11-30', '789 Pine Rd, Izmir', 'Technician');
18
19 -- Basic select
20 SELECT * FROM employees;
```

SELECTING COLUMNS

- Using **SELECT *** in application code (Python, Java, PHP) is poor practice. It fetches all columns, which can hurt **database performance** by retrieving more data than needed and **application performance** by increasing server traffic, slowing responses, and reducing scalability. Instead, explicitly list only the needed columns for more efficient queries. Reserve **SELECT *** for ad-hoc data inspection.
- The following query uses the **SELECT** statement to retrieve first name, last name, and email of customers from the customer table:

```
SELECT
    first_name,
    last_name,
    email
FROM
    customer;
```



Exercise: Now on SQL Studio, select only ssn, birthdate and address of the employees

SELECT statement with expressions

- The following example uses the SELECT statement to return the full names and emails of all customers from the customer table:

```
SELECT
    first_name || ' ' || last_name,
    email
FROM
    customer;
```

Output:

?column?		email
-----+		
Jared Ely		jared.ely@example.com
Mary Smith		mary.smith@example.com
Patricia Johnson		patricia.johnson@example.com
...		

SELECT statement with expressions

- In the previous example, we used the **concatenation** operator `||` to concatenate the first name, space, and last name of every customer.
- Notice the first column of the output doesn't have a name but `?column?`. To assign a name to a column temporarily in the query, you can use a column alias:

`expression AS column_alias`

- The `AS` keyword is optional. Therefore, you can use a shorter syntax:

`expression column_alias`

SELECT statement with expressions

For example, you can assign a column alias `full_name` to the first column of the query as follows:

```
SELECT
    first_name || ' ' || last_name full_name,
    email
FROM
    customer;
```



Output:

```
full_name      | email
-----+-----
Jared Ely      | jared.ely@example.com
Mary Smith     | mary.smith@example.com
Patricia Johnson | patricia.johnson@example.com
...
```



SELECT statement without a FROM clause

The `FROM` clause of the `SELECT` statement is optional. Therefore, you can omit it in the `SELECT` statement.

Typically, you use the `SELECT` clause with a function to retrieve the function result. For example:

```
SELECT NOW();
```

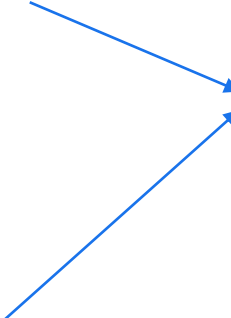
In this example, we use the `NOW()` function in the `SELECT` statement. It'll return the current date and time of the PostgreSQL server.

SELECT EXAMPLES

```
SELECT
  first_name,
  last_name AS surname
FROM customer;
```

```
SELECT
  first_name,
  last_name surname
FROM customer;
```

THESE TWO
EXAMPLES ARE
IDENTICAL



```
SELECT
  first_name || ' ' || last_name "full name"
FROM
  customer;
```

USE QUOTES WHEN
YOU HAVE SPACE
BETWEEN
ASSIGNED
COLUMN NAMES



ORDER BY COMMAND



PostgreSQL ORDER BY clause

When you query data from a table, the `SELECT` statement returns rows in an unspecified order. To sort the rows of the result set, you use the `ORDER BY` clause in the `SELECT` statement.

The `ORDER BY` clause allows you to sort rows returned by a `SELECT` clause in ascending or descending order based on a sort expression.

The following illustrates the syntax of the `ORDER BY` clause:

```
SELECT
  select_list
FROM
  table_name
ORDER BY
  sort_expression1 [ASC | DESC],
  sort_expression2 [ASC | DESC],
  ...;
```

PostgreSQL ORDER BY clause

IN THE PREVIOUS SYNTAX:

- First, specify a sort expression, which can be a column or an expression, that you want to sort after the `ORDER BY` keywords. If you want to sort the result set based on multiple columns or expressions, you need to place a comma (,) between two columns or expressions to separate them.
- Second, you use the `ASC` option to sort rows in ascending order and the `DESC` option to sort rows in descending order. If you omit the `ASC` or `DESC` option, the `ORDER BY` uses `ASC` by default.

PostgreSQL ORDER BY clause

PostgreSQL evaluates the clauses in the `SELECT` statement in the following order: `FROM`, `SELECT`, and `ORDER BY`:



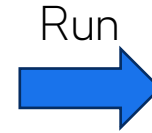
Due to the order of evaluation, if you have a column alias in the `SELECT` clause, you can use it in the `ORDER BY` clause.

PostgreSQL ORDER BY clause

customer	
* customer_id	
store_id	
first_name	
last_name	
email	
address_id	
activebool	
create_date	
last_update	
active	



```
SELECT
    first_name,
    last_name
FROM
    customer
ORDER BY
    first_name ASC;
```



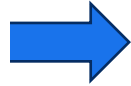
	first_name character varying (45)	last_name character varying (45)
1	Aaron	Selby
2	Adam	Gooch
3	Adrian	Clary
4	Agnes	Bishop
5	Alan	Kahn
6	Albert	Crouse
7	Alberto	Henning
8	Alex	Gresham
9	Alexander	Fennell
10	Alfred	Casillas
11	Alfredo	Mcadams
12	Alice	Stewart
13	Alicia	Mills

Given a customer table

Result in ASC Form

PostgreSQL ORDER BY clause-exercise

employees		
PK	varchar	ssn
	varchar	fname
	bpchar	minit
	varchar	lname
	date	birth_date
	text	address
	varchar	job_type



Write SQL
Command to
retrieve
following
result using
ASC,
concatenation,
column
selection



Run

ssn	name	birth_date	
555-00-1212	Robert Taylor	1978-11-30T00:00:00.000Z	
987-65-4321	John Smith	1985-01-15T00:00:00.000Z	
123-45-6789	Alice Johnson	1992-05-22T00:00:00.000Z	

Expected Result

Given the employees table

EXAMPLES

```
SELECT
  first_name,
  last_name
FROM
  customer
ORDER BY
  last_name DESC;
```

This statement selects the first name and last name from the customer table and sorts the rows by values in the last name column in descending order.

```
SELECT
  first_name,
  last_name
FROM
  customer
ORDER BY
  first_name ASC,
  last_name DESC;
```

This statement selects the first name and last name from the customer table and sorts the rows by the first name initially in ascending order and last name afterwards in descending order.

```
SELECT
  first_name,
  LENGTH(first_name) len
FROM
  customer
ORDER BY
  len DESC;
```

The `LENGTH()` function accepts a string and returns the length of that string. The above statement selects the first names and their lengths. It sorts the rows by the lengths of the first names:

Exercise: Let's quickly apply this to employee table

EXAMPLES

```
SELECT
    first_name,
    last_name
FROM
    customer
ORDER BY
    last_name DESC;
```

This statement selects the first name and last name from the customer table and sorts the rows by values in the last name column in descending order.

```
SELECT
    first_name,
    last_name
FROM
    customer
ORDER BY
    first_name ASC,
    last_name DESC;
```

This statement selects the first name and last name from the customer table and sorts the rows by the first name initially in ascending order and last name afterwards in descending order.

```
SELECT
    first_name,
    LENGTH(first_name) len
FROM
    customer
ORDER BY
    len DESC;
```

The `LENGTH()` function accepts a string and returns the length of that string. The above statement selects the first names and their lengths. It sorts the rows by the lengths of the first names:

Exercise: Let's quickly apply this to employee table on SQL Studio

PostgreSQL ORDER BY clause and NULL

In the database world, `NULL` is a marker that indicates the missing data or the data is unknown at the time of recording.

When you sort rows that contain `NULL`, you can specify the order of `NULL` with other non-null values by using the `NULLS FIRST` or `NULLS LAST` option of the `ORDER BY` clause:

```
ORDER BY sort_expression [ASC | DESC] [NULLS FIRST | NULLS LAST]
```



The `NULLS FIRST` option places `NULL` before other non-null values and the `NULLS LAST` option places `NULL` after other non-null values.

PostgreSQL ORDER BY clause and NULL

```
-- create a new table
CREATE TABLE sort_demo(num INT);

-- insert some data
INSERT INTO sort_demo(num)
VALUES
  (1),
  (2),
  (3),
  (null);
```

```
SELECT
  num
FROM
  sort_demo
ORDER BY
  num NULLS LAST;
```



```
num
-----
  1
  2
  3
null
(4 rows)
```

Same as default

Given this table let's run some SQL commands

```
SELECT
  num
FROM
  sort_demo
ORDER BY
  num;
```



```
num
-----
  1
  2
  3
null
(4 rows)
```

Default sort places
null at the bottom

```
SELECT
  num
FROM
  sort_demo
ORDER BY
  num NULLS FIRST;
```



```
num
-----
null
  1
  2
  3
(4 rows)
```

PostgreSQL ORDER BY clause and NULL

```
-- create a new table
CREATE TABLE sort_demo(num INT);

-- insert some data
INSERT INTO sort_demo(num)
VALUES
  (1),
  (2),
  (3),
  (null);
```

Given this table Run and find the output of the following SQL commands

```
SELECT
  num
FROM
  sort_demo
ORDER BY
  num DESC;
```



?

```
SELECT
  num
FROM
  sort_demo
ORDER BY
  num DESC NULLS LAST;
```



?

DISTINCT COMMAND



PostgreSQL SELECT DISTINCT

The `SELECT DISTINCT` removes duplicate rows from a result set. The `SELECT DISTINCT` clause retains one row for each group of duplicates.

The `SELECT DISTINCT` clause can be applied to one or more columns in the select list of the `SELECT` statement.

The following illustrates the syntax of the `DISTINCT` clause:

```
SELECT
  DISTINCT column1
FROM
  table_name;
```

PostgreSQL SELECT DISTINCT

If you specify multiple columns, the `SELECT DISTINCT` clause will evaluate the duplicate based on the combination of values in these columns. For example:

```
SELECT
    DISTINCT column1, column2
FROM
    table_name;
```

In this syntax, the `SELECT DISTINCT` uses the combination of values in both `column1` and `column2` columns for evaluating the duplicate.

PostgreSQL SELECT DISTINCT

If you specify multiple columns, the `SELECT DISTINCT` clause will evaluate the duplicate based on the combination of values in these columns. For example:

```
SELECT
    DISTINCT column1, column2
FROM
    table_name;
```

In this syntax, the `SELECT DISTINCT` uses the combination of values in both `column1` and `column2` columns for evaluating the duplicate.



If you want to find distinct values of all columns in a table, you can use

```
SELECT DISTINCT *:
```

PostgreSQL SELECT DISTINCT-Examples

```
CREATE TABLE colors(  
  id SERIAL PRIMARY KEY,  
  bcolor VARCHAR,  
  fcolor VARCHAR  
);
```

```
INSERT INTO  
  colors (bcolor, fcolor)  
VALUES  
  ('red', 'red'),  
  ('red', 'red'),  
  ('red', NULL),  
  (NULL, 'red'),  
  (NULL, NULL),  
  ('green', 'green'),  
  ('blue', 'blue'),  
  ('blue', 'blue');
```

Create a table and insert following data for practice.
Then check out with a select statement if your data has
been properly inserted.

```
SELECT  
  id,  
  bcolor,  
  fcolor  
FROM  
  colors;
```



id	bcolor	fcolor
1	red	red
2	red	red
3	red	null
4	null	red
5	null	null
6	green	green
7	blue	blue
8	blue	blue

(8 rows)

PostgreSQL SELECT DISTINCT-Examples

Run following commands

```
SELECT
  DISTINCT bcolor
FROM
  colors
ORDER BY
  bcolor;
```

Single column

```
SELECT
  DISTINCT bcolor, fcolor
FROM
  colors
ORDER BY
  bcolor,
  fcolor;
```

Multiple Column

```
SELECT DISTINCT
  rental_rate
FROM
  film
ORDER BY
  rental_rate;
```

DISTINCT AND
ORDER BY together

- **Note** that PostgreSQL treats NULLs as duplicates so that it keeps one NULL for all NULLs when you apply the SELECT DISTINCT clause.
- We often use the SELECT DISTINCT clause to analyse the uniqueness of values in a column and then we can apply ORDER BY to a column that we applied DISTINCT. See the third example above.

PRACTICE

Create following table and insert the data then try to write SQL commands and retrieve the data for the next slides..

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department VARCHAR(50),  
    city VARCHAR(50),  
    salary NUMERIC(10,2),  
    bonus NUMERIC(10,2),  
    hire_date DATE  
);
```

```
INSERT INTO employees (first_name, last_name, department, city, salary, bonus, hire_date)  
VALUES  
( 'Ali', 'Yilmaz', 'IT', 'Ankara', 45000, 5000, '2021-03-15'),  
( 'Ayse', 'Demir', 'HR', 'Istanbul', 42000, 4000, '2020-06-10'),  
( 'Mehmet', 'Kaya', 'IT', 'Izmir', 47000, 4500, '2022-01-20'),  
( 'Zeynep', 'Acar', 'Finance', 'Ankara', 52000, 6000, '2019-11-05'),  
( 'Can', 'Oz', 'HR', 'Bursa', 40000, 3500, '2023-04-12'),  
( 'Elif', 'Arslan', 'IT', 'Istanbul', 48000, 5500, '2021-09-01'),  
( 'Burak', 'Celik', 'Finance', 'Antalya', 53000, 7000, '2018-07-19'),  
( 'Fatma', 'Sahin', 'Marketing', 'Ankara', 39000, 3000, '2022-05-25'),  
( 'Murat', 'Eren', 'Marketing', 'Izmir', 41000, 3200, '2020-02-14'),  
( 'Selin', 'Kurt', 'IT', 'Istanbul', 50000, 6500, '2017-12-30');|
```

PRACTICE

Create following table and insert the data then try to write SQL commands and retrieve the data for the next slides..

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50),  
    department VARCHAR(50), city VARCHAR(50), salary NUMERIC(10,2), bonus NUMERIC(10,2),  
    hire_date DATE  
);
```

```
INSERT INTO employees (first_name, last_name, department, city, salary,  
bonus, hire_date)  
VALUES  
( 'Ali', 'Yilmaz', 'IT', 'Ankara', 45000, 5000, '2021-03-15'),  
( 'Ayse', 'Demir', 'HR', 'Istanbul', 42000, 4000, '2020-06-10'),  
( 'Mehmet', 'Kaya', 'IT', 'Izmir', 47000, 4500, '2022-01-20'),  
( 'Zeynep', 'Acar', 'Finance', 'Ankara', 52000, 6000, '2019-11-05'),  
( 'Can', 'Oz', 'HR', 'Bursa', 40000, 3500, '2023-04-12'),  
( 'Elif', 'Arslan', 'IT', 'Istanbul', 48000, 5500, '2021-09-01'),  
( 'Fatma', 'Sahin', 'Marketing', 'Ankara', 39000, 3000, '2022-05-25'),  
( 'Murat', 'Eren', 'Marketing', 'Izmir', 41000, 3200, '2020-02-14'),  
( 'Selin', 'Kurt', 'IT', 'Istanbul', 50000, 6500, '2017-12-30');
```

PRACTICE

- 1- Retrieve each employee's full name as `employee_name` and their annual salary as `annual_salary`.
- 2- Retrieve each employee's full name, department, and total compensation (salary + bonus) as `total_compensation`.
- 3-Retrieve each employee's full name and their salary after a 15% increase as `increased_salary`, ordered by the increased salary from highest to lowest.
- 4-Retrieve all distinct department-city combinations, ordered first by department and then by city.
- 5-Retrieve the top 3 employees with the highest total compensation (salary + bonus), showing full name and total compensation.
- 6-Retrieve all distinct cities, ordered in reverse alphabetical order.

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM